Agentic Delivery Control Tower

Junior FDE Pre-screening Assignment · Sidharth Jain · May 2026

Live demo huggingface.co/spaces/sidjain204/dct-prod GitHub Robomineboy/delivery-control-tower

1. MULTI-AGENT ARCHITECTURE

The system is a governed multi-agent pipeline of seven specialized agents operating over a single shared typed state object (Pydantic v2). An orchestrator routes each query into one of three conditional paths — write, risk, or summary — based on detected intent. A pre-orchestration validation layer (injection detection, input sanitization, role check) runs before any agent is invoked.

Agent	Responsibility
Intake (LLM)	Parses natural language into structured intent: project, action type (read/write), urgency, and search filters.
Retrieval	Semantic FAISS search over live Jira tickets with hybrid structured field filtering on status, assignee, and priority.
Risk Analysis (LLM)	Detects cross-ticket patterns: blockers, SLA violations, overloaded assignees, unowned criticals. Returns findings with severity (CRITICAL → LOW) and confidence (0.0–1.0).
Critic (rule-based)	Validates each finding against retrieved ticket IDs. Removes unsupported claims, caps confidence on single-ticket evidence. Deterministic and fully auditable.
Planning (LLM)	Converts findings into prioritized recommendations (IMMEDIATE / THIS_WEEK / BACKLOG). On the write path, generates exact Jira operations matching the user's request only.
Security (rule-based)	Validates write actions before the approval gate: ticket ID format, valid team member names, date format, comment length.
Writer	Executes approved Jira actions via REST API (write:jira-work scope). Restricted to four typed tools: create_ticket, update_assignee, set_due_date, add_comment.

2. SECURITY, SAFETY, AND GUARDRAILS

- **Pre-orchestration layer —** Prompt injection detection, input sanitization, role validation, and size constraints run before the agent graph initializes. Adversarial input never enters an LLM.
- **Confidence thresholds —** MIN_RETRIEVAL, MIN_PLANNING, and MIN_CRITIC thresholds are externalized in config. If retrieval confidence falls below threshold, downstream agents are blocked from generating recommendations — the system degrades gracefully rather than hallucinating on sparse data.
- **Critic as hallucination guard —** Every recommendation must be traceable to a specific ticket ID in the retrieved evidence set. Unsupported claims are removed before output.
- **Security Agent for write actions —** All Jira mutations are validated against a rule set before reaching the human approval gate. Malformed or out-of-scope actions are blocked and logged with a reason.
- **Human-in-the-loop approval —** No Jira mutation occurs without explicit user approval. Write actions are presented as a per-action review card. Unapproved actions are discarded and audit-logged. This is an architectural prerequisite for enterprise deployment, not a feature.
- **Typed tool access only —** The Writer Agent cannot issue arbitrary API calls. Four typed functions with explicit signatures are the complete surface area — fully auditable and testable.
- **Token visibility —** Per-agent input/output token counts and estimated cost are surfaced in the UI on every query, providing the foundation for prompt optimization and client cost transparency.

3. IMPLEMENTATION

Stack. FastAPI (Python 3.10) · Groq llama-3.1-8b-instant (primary), OpenAI GPT-4o (fallback) · FAISS + sentence-transformers (all-MiniLM-L6-v2) · Jira REST API via OAuth2 three-legged flow with silent refresh token rotation · Pydantic v2 for typed schemas at every agent boundary · Vanilla HTML/CSS/JS frontend with live agent trace and dev console · Deployed on HuggingFace Spaces (Docker).

Coordination. Agents are instantiated as stateless Python classes. Each receives the shared PipelineState, appends its typed output, and returns. The orchestrator determines which agents fire based on intent and confidence gates. Write-path pipelines store pending actions in memory keyed by run_id until the approval endpoint fires.

Resilience. Retrieval falls back to unfiltered semantic search when structured filters return no results. The Jira loader falls back to a curated ground truth dataset on auth failure, ensuring the pipeline always produces valid output. OAuth refresh tokens expire after 90 days with automatic re-authorization.

4. USE OF AI / LLMS AND AGENT COLLABORATION

LLMs are used where flexible reasoning is required: intent parsing, cross-ticket risk analysis, natural language summarization, and action planning. Rule-based agents handle validation where determinism and auditability are non-negotiable. This is a deliberate architectural separation — the system is a typed pipeline with LLM reasoning at specific bounded nodes, not an LLM workflow.

The key collaboration pattern is the Critic validating Risk and Planning outputs against the original retrieved evidence. This agent-reviews-agent step catches hallucinations that are structurally impossible to detect in a single-agent system. The Planning Agent similarly cannot generate write actions beyond what the user explicitly requested, enforced through prompt constraints verified by the Security Agent.

The trade-off between autonomy and control is explicit by design. Read-path queries run fully autonomously. Write-path queries require human approval at the boundary between recommendation and execution. The system is built to augment delivery manager judgment, not replace it.

5. EVALUATION — GOLD LABEL QUERIES

Five representative queries were used to validate end-to-end system behavior. Each was manually verified against expected intent classification, retrieval results, agent path, and output correctness.

Query	Expected intent	Expected path	Verification criterion
"Show me blocked tickets"	blocker_analysis	Risk	SS-15 (Blocked) surfaces as top result; Risk Agent flags blocker with evidence; Critic validates ticket ID present.
"What is Aryan working on?"	person_query	Summary	assignee_contains filter applied; 3 Aryan tickets retrieved; Summary Agent answers directly without risk analysis.
"What are the biggest risks this sprint?"	risk_analysis	Risk	2–4 findings generated with severity and confidence; all findings backed by ticket IDs; Critic passes or flags with reason.
"Assign SS-10 to Aryan Ghosh"	write_action	Write	Exactly one update_assignee action generated; Security Agent clears it; HITL approval card shown; Jira updates on approval.
"Overall project health?"	project_summary	Summary	All 10 tickets fetched; Summary Agent produces honest assessment referencing specific ticket IDs and statuses.